

Real-Time Examples of the Fluent Interface Design Pattern in C#

I will discuss the **Real-Time Examples of the Fluent Interface Design Pattern in C#** in this article. Please read our previous article discussing the basic concepts of the [Fluent Interface Design Pattern in C#](#) with Examples. At the end of this article, you will understand the following Real-time Examples using the Fluent Interface Design Pattern in C#.

1. [Automapper Implementation using Fluent Interface](#)
2. [FluentValidation](#)
3. [RestSharp for API Requests](#)
4. [Fluent SQL Query Builder](#)
5. [Fluent HTML Tag Builder](#)
6. [Fluent Shopping Cart Builder](#)
7. [Fluent User Profile Builder](#)
8. [Fluent Game Character Builder](#)
9. [Fluent Pizza Builder](#)
10. [Entity Framework Code First Model Properties Configuration](#)
11. [Fluent Email Builder](#)

Real-Time Example of Fluent Interface Design Pattern in C#: Automapper

Automapper is a library used to map objects in .NET applications, and it's particularly known for its concise and readable configuration using a fluent interface. Here's an example demonstrating Automapper's fluent configuration and usage: First, install the Automapper package from NuGet.

Setting up Source and Destination Classes:

```
namespace FluentInterfaceDesignPattern
{
    public class Customer
    {
        public string FirstName { get; set; }
        public string LastName { get; set; }
    }
    public class CustomerDTO
    {
        public string FullName { get; set; }
    }
}
```

In this case, we want to map a Customer object to a CustomerDTO object, where the FullName in CustomerDTO should combine FirstName and LastName from Customer.

Configuring the mapping using Automapper's Fluent Interface:

```
using AutoMapper;
namespace FluentInterfaceDesignPattern
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            //Provide all the Mapping Configuration
        }
    }
}
```

```

var config = new MapperConfiguration(cfg =>
{
    //Configuring Employee and EmployeeDTO
    cfg.CreateMap<Customer, CustomerDTO>()
        .ForMember(dest => dest.FullName,
            opt => opt.MapFrom(src => $"{src.FirstName}
{src.LastName}"));
    //Any Other Mapping Configuration ....
});

//Create an Instance of Mapper and return that Instance
var mapper = new Mapper(config);
return mapper;
}
}
}

```

In the configuration part:

- **CreateMap<Customer, CustomerDTO>()** sets up a mapping between the Customer and CustomerDTO types.
- **ForMember** is used to configure how individual properties are mapped. In this case, we're using it to define how the FullName property should be populated.

Performing the Mapping:

```

using System;
namespace FluentInterfaceDesignPattern
{
    //Testing the Fluent Interface Design Pattern
    public class Program
    {
        public static void Main()
        {
            var customer = new Customer { FirstName = "John", LastName =
"Doe" };

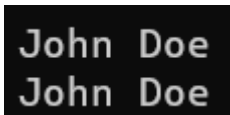
            //Initializing AutoMapper
            var mapper = MapperConfig.InitializeAutomapper();
            //Way1: Specify the Destination Type and to The Map Method pass
the Source Object
            var customerDTO1 = mapper.Map<CustomerDTO>(customer);
            Console.WriteLine(customerDTO1.FullName); // Outputs: John Doe
            //Way2: Specify the both Source and Destination Type
            //and to The Map Method pass the Source Object
            var customerDTO2 = mapper.Map<Customer, CustomerDTO>(customer);
            Console.WriteLine(customerDTO2.FullName); // Outputs: John Doe

            Console.ReadKey();
        }
    }
}

```

This is a simple example, but you might have multiple, more complex objects and relationships in real-world scenarios. AutoMapper's fluent interface allows for a highly readable and compact definition of these mappings.

When you run the above code, you will get the following output.



Real-Time Example of Fluent Interface Design Pattern in C#: FluentValidation

Let's look at the FluentValidation library, a popular choice for implementing validation logic in .NET applications. FluentValidation uses the Fluent Interface design pattern, allowing you to define validation rules in a chained, readable manner. Let us see how we can implement the above example using the Fluent Interface Design Pattern in C#:

Setting up a simple class:

Suppose we have a Person class that we want to validate.

```
namespace FluentInterfaceDesignPattern
{
    public class Person
    {
        public string FirstName { get; set; }
        public string LastName { get; set; }
        public int Age { get; set; }
    }
}
```

Setting up FluentValidation:

Firstly, ensure you have the FluentValidation NuGet package installed. Now, let's create a validator for the Person class.

```
using FluentValidation;
namespace FluentInterfaceDesignPattern
{
    public class PersonValidator : AbstractValidator<Person>
    {
        public PersonValidator()
        {
            RuleFor(x => x.FirstName)
                .NotEmpty().WithMessage("Please specify a first name")
                .Length(0, 10).WithMessage("First name cannot be longer than 10 characters");
            RuleFor(x => x.LastName)
                .NotEmpty().WithMessage("Please specify a last name");
            RuleFor(x => x.Age)
                .InclusiveBetween(0, 150).WithMessage("Age must be between 0 and 150");
        }
    }
}
```

In the PersonValidator:

- **RuleFor(x => x.FirstName)** starts the rule definition for the FirstName property.
- **NotEmpty()** ensures that the property isn't empty.
- **WithMessage("...")** allows you to specify a custom error message.
- Chaining methods like **.Length(0, 10)** allows you to apply multiple validations fluently.

Using the Validator:

With the validator defined, we can now validate instances of the Person class.

```
using System;
namespace FluentInterfaceDesignPattern
{
    //Testing the Fluent Interface Design Pattern
    public class Program
    {
        public static void Main()
        {
            var person = new Person
            {
                FirstName = "Johnathon",
                LastName = "",
                Age = 200
            };
            var validator = new PersonValidator();
            var results = validator.Validate(person);
            if (!results.IsValid)
            {
                foreach (var failure in results.Errors)
                {
                    Console.WriteLine($"Property {failure.PropertyName}
failed validation. Error was: {failure.ErrorMessage}");
                }
            }

            Console.ReadKey();
        }
    }
}
```

FluentValidation provides a clear, expressive, and fluent way to define and execute validation logic, making it a real-time example of the Fluent Interface design pattern in the C# Ecosystem. When you run the above code, you will get the following output.

```
Property LastName failed validation. Error was: Please specify a last name
Property Age failed validation. Error was: Age must be between 0 and 150
```

Real-Time Example of Fluent Interface Design Pattern in C#: RestSharp for API Requests

Let's understand another real-time example by considering the popular RestSharp library. RestSharp is a simple, open-source REST client for .NET platforms. It uses the Fluent Interface pattern to create and execute REST requests in a readable and expressive way. Setting up the library: First, ensure you have installed the RestSharp NuGet package.

Using RestSharp with Fluent Interface: Suppose we want to make a GET request to retrieve a user from the JSONPlaceholder API.

```

using System;
using RestSharp;
namespace FluentInterfaceDesignPattern
{
    public class User
    {
        public int Id { get; set; }
        public string Name { get; set; }
        //... Other properties
    }
    //Testing the Fluent Interface Design Pattern
    public class Program
    {
        public static void Main()
        {
            var client = new
RestClient("https://jsonplaceholder.typicode.com");
            var request = new RestRequest("users/{id}", Method.Get)
                .AddUrlSegment("id", 1);
            RestResponse<User> response = client.Execute<User>(request);
            if (response.StatusCode == System.Net.HttpStatusCode.OK)
            {
                Console.WriteLine($"User ID: {response.Data.Id}, User Name:
{response.Data.Name}");
            }
            else
            {
                Console.WriteLine($"Error: {response.StatusDescription}");
            }
            Console.ReadKey();
        }
    }
}

```

In the example above:

- We begin by creating a RestClient pointing to the base URL of the API.
- For the request creation, we instantiate RestRequest and specify the resource endpoint as “users/{id}” and the method as GET.
- .AddUrlSegment(“id”, 1) is a fluent method that replaces the {id} segment in the URL with the provided value 1.
- We then execute the request and get a typed response back using the client.Execute<User>(request).

This fluent construction of the request makes the code easy to read and understand, even if further parameters, headers, or configurations are added. RestSharp’s ability to configure and execute requests fluently makes it a good real-world example of the Fluent Interface pattern in C#. When you run the above code, you will get the following output.

User ID: 1, User Name: Leanne Graham

Real-Time Example of Fluent Interface Design Pattern in C#: Fluent SQL Query Builder

We’ll focus on a common scenario in software development: building and executing SQL queries. Libraries like Dapper FluentMap and QueryBuilder often employ Fluent Interfaces for such tasks. However, for the sake of simplicity, let’s build a basic example from scratch. Imagine you’re designing a simple Fluent SQL Query Builder to create SELECT statements.

Setting up the QueryBuilder Class:

```
using System;
namespace FluentInterfaceDesignPattern
{
    public class QueryBuilder
    {
        private string _select;
        private string _from;
        private string _where;
        public QueryBuilder Select(string columns)
        {
            _select = $"SELECT {columns}";
            return this;
        }
        public QueryBuilder From(string table)
        {
            _from = $"FROM {table}";
            return this;
        }
        public QueryBuilder Where(string condition)
        {
            _where = $"WHERE {condition}";
            return this;
        }
        public string Build()
        {
            return $"{_select} {_from} {_where}";
        }
    }
}
```

Using the Fluent QueryBuilder:

Let's craft an SQL statement using our Fluent QueryBuilder:

```
using System;
namespace FluentInterfaceDesignPattern
{
    //Testing the Fluent Interface Design Pattern
    public class Program
    {
        public static void Main()
        {
            var query = new QueryBuilder()
                .Select("Name, Age")
                .From("Users")
                .Where("Age > 21")
                .Build();
            Console.WriteLine(query); // Outputs: SELECT Name, Age FROM
Users WHERE Age > 21
            Console.ReadKey();
        }
    }
}
```

```
}
```

This example illustrates how the Fluent Interface design pattern can make code more readable and intuitive. Instead of concatenating strings or using a complex set of nested functions, developers can chain method calls to mirror the natural order and structure of SQL queries.

This example is a very basic version of what real-world libraries offer. In practice, libraries like Dapper FluentMap, for instance, have a vast range of functionalities, such as handling relationships between tables, more complex WHERE conditions, JOIN operations, etc. However, the core principle remains the same: provide the developer with a fluid, intuitive interface.

Real-Time Example of Fluent Interface Design Pattern in C#: Fluent HTML Tag Builder

Let's consider a practical example of dynamically building HTML tags in a C# application. This example illustrates the usage of the Fluent Interface Design Pattern to fluently define HTML tags, their attributes, and their inner content. Let us see how we can implement the above example using the Fluent Interface Design Pattern in C#:

HTML Tag Builder Class:

```
using System.Linq;
using System.Collections.Generic;
namespace FluentInterfaceDesignPattern
{
    public class HtmlTagBuilder
    {
        private readonly string _tagName;
        private string _content;
        private Dictionary<string, string> _attributes = new
Dictionary<string, string>();
        public HtmlTagBuilder(string tagName)
        {
            _tagName = tagName;
        }
        public HtmlTagBuilder WithContent(string content)
        {
            _content = content;
            return this;
        }
        public HtmlTagBuilder AddAttribute(string attribute, string value)
        {
            _attributes[attribute] = value;
            return this;
        }
        public override string ToString()
        {
            var attributes = string.Join(" ", _attributes.Select(a => $"
{a.Key}='{a.Value}'"));
            return $"<{_tagName} {attributes}>{_content}</{_tagName}>";
        }
    }
}
```

In this example:

- The `HtmlTagBuilder` class provides a fluent interface to define an HTML tag. We start by specifying the tag name ("div") in the constructor.
- The `WithContent` method is used to set the inner content of the tag and returns the same `HtmlTagBuilder` instance for further chaining.
- The `AddAttribute` method adds attributes like "class" and "id", returning the same instance for method chaining.
- Finally, the `ToString` method generates the final HTML tag string.

Here is how you can use the `HtmlTagBuilder` class to construct an HTML tag fluently:

```
using System;
namespace FluentInterfaceDesignPattern
{
    //Testing the Fluent Interface Design Pattern
    public class Program
    {
        public static void Main()
        {
            var tag = new HtmlTagBuilder("div")
                .AddAttribute("class", "container")
                .AddAttribute("id", "mainContainer")
                .WithContent("Hello, world!")
                .ToString();
            Console.WriteLine(tag);
            // Outputs: <div class='container' id='mainContainer'>Hello,
world!</div>
            Console.ReadKey();
        }
    }
}
```

Here, the Fluent Interface Design Pattern makes the code readable and easy to understand while allowing a complex object (HTML tag with attributes and content) to be built straightforwardly.

Real-Time Example of Fluent Interface Design Pattern in C#: Fluent Shopping Cart Builder

Let's look at a different scenario. This time, let's consider the construction of a product order with a shopping cart, which is common in e-commerce systems. Let us see how we can implement the above example using the Fluent Interface Design Pattern in C#:

Core Classes:

```
using System.Linq;
using System.Collections.Generic;
namespace FluentInterfaceDesignPattern
{
    public class Product
    {
        public string Name { get; set; }
        public decimal Price { get; set; }
    }
    public class Order
    {
        public List<Product> Products { get; } = new List<Product>();
        public decimal TotalPrice => Products.Sum(p => p.Price);
    }
}
```



```

        public Order AddProduct(Product product)
        {
            Products.Add(product);
            return this;
        }
        public Order RemoveProduct(Product product)
        {
            Products.Remove(product);
            return this;
        }
    }
}

```

Fluent Shopping Cart Builder:

```

using System.Linq;
namespace FluentInterfaceDesignPattern
{
    public class ShoppingCartBuilder
    {
        private readonly Order _order = new Order();
        public ShoppingCartBuilder AddProduct(string name, decimal price)
        {
            var product = new Product { Name = name, Price = price };
            _order.AddProduct(product);
            return this;
        }
        public ShoppingCartBuilder RemoveProduct(string name)
        {
            var product = _order.Products.FirstOrDefault(p => p.Name ==
name);
            if (product != null)
            {
                _order.RemoveProduct(product);
            }
            return this;
        }
        public Order Checkout()
        {
            return _order;
        }
    }
}

```

Usage:

Here's how you can fluently create an order with the ShoppingCartBuilder:

```

using System;
namespace FluentInterfaceDesignPattern
{
    //Testing the Fluent Interface Design Pattern
    public class Program
    {

```

```

    public static void Main()
    {
        var order = new ShoppingCartBuilder()
            .AddProduct("Laptop", 1000m)
            .AddProduct("Mouse", 50m)
            .RemoveProduct("Mouse")
            .AddProduct("Keyboard", 70m)
            .Checkout();
        Console.WriteLine($"Total Price: {order.TotalPrice}");
        // Outputs: Total Price: 1070
        Console.ReadKey();
    }
}

```

In this example:

- The ShoppingCartBuilder provides a fluent interface to construct an order.
- The AddProduct method allows adding a product to the order by specifying its name and price. It returns the same ShoppingCartBuilder instance for further chaining.
- The RemoveProduct method allows removing a product from the order by its name, again returning the same instance for method chaining.
- The Checkout method completes the order and returns the constructed Order object.

This approach offers a readable way to build an order by chaining methods, making it intuitive to add and remove products. The Fluent Interface Design Pattern helps in providing a clean and organized way to build complex objects step by step.

Real-Time Example of Fluent Interface Design Pattern in C#: Fluent User Profile Builder

Let's consider another example, this time in the context of configuring a complex user profile system. Let us see how we can implement the above example using the Fluent Interface Design Pattern in C#:

Core Classes:

```

using System.Collections.Generic;
namespace FluentInterfaceDesignPattern
{
    public class UserProfile
    {
        public string Name { get; set; }
        public string Email { get; set; }
        public string Bio { get; set; }
        public List<string> Interests { get; private set; } = new
List<string>();
        public Dictionary<string, string> SocialLinks { get; private set; }
= new Dictionary<string, string>();
    }
}

```

Fluent User Profile Builder:

```

namespace FluentInterfaceDesignPattern
{
    public class UserProfileBuilder

```

```

{
    private readonly UserProfile _profile = new UserProfile();
    public UserProfileBuilder Named(string name)
    {
        _profile.Name = name;
        return this;
    }
    public UserProfileBuilder WithEmail(string email)
    {
        _profile.Email = email;
        return this;
    }
    public UserProfileBuilder WithBio(string bio)
    {
        _profile.Bio = bio;
        return this;
    }
    public UserProfileBuilder AddInterest(string interest)
    {
        _profile.Interests.Add(interest);
        return this;
    }
    public UserProfileBuilder WithSocialLink(string platform, string
link)
    {
        _profile.SocialLinks[platform] = link;
        return this;
    }
    public UserProfile Build()
    {
        return _profile;
    }
}
}

```

Usage:

Here's how you can fluently create a user profile with the UserProfileBuilder:

```

using System;
namespace FluentInterfaceDesignPattern
{
    //Testing the Fluent Interface Design Pattern
    public class Program
    {
        public static void Main()
        {
            var userProfile = new UserProfileBuilder()
                .Named("Pranaya Rout")
                .WithEmail("pranaya@dotnettutorials.com")
                .WithBio("Software Developer at XYZ.")
                .AddInterest("Coding")
                .AddInterest("Hiking")
                .WithSocialLink("Twitter", "https://twitter.com/RoutPranaya")

```

```

        .WithSocialLink("LinkedIn",
"https://www.linkedin.com/in/pranaya-rout/")
        .Build();
        Console.WriteLine($"Name: {userProfile.Name}");
        Console.WriteLine($"Bio: {userProfile.Bio}");
        Console.WriteLine($"Interests: {string.Join(", ",
userProfile.Interests)}");
        // And so on...
        Console.ReadKey();
    }
}
}

```

In this example:

- The UserProfileBuilder provides a fluent interface to set up a user profile.
- The methods Named, WithEmail, WithBio, etc., configure different aspects of the user profile, and each returns the builder instance to facilitate further chaining.
- The Build method completes the profile setup and returns the constructed UserProfile object.

With this fluent approach, it's clear and concise to set up complex objects. The Fluent Interface Design Pattern ensures that the code remains highly readable, especially when there are multiple attributes or configurations to set.

Real-Time Example of Fluent Interface Design Pattern in C#: Fluent Game Character Builder

Let's explore the Fluent Interface pattern in the context of building a configuration for a game character. This builder pattern can be seen in game development, where a character's attributes, equipment, and skills are set during character creation. Let us see how we can implement the above example using the Fluent Interface Design Pattern in C#:

Core Classes:

```

using System.Collections.Generic;
namespace FluentInterfaceDesignPattern
{
    public class GameCharacter
    {
        public string Name { get; set; }
        public int Health { get; set; }
        public string Weapon { get; set; }
        public List<string> Skills { get; private set; } = new List<string>
();
    }
}

```

Fluent Character Builder:

```

namespace FluentInterfaceDesignPattern
{
    public class CharacterBuilder
    {
        private readonly GameCharacter _character = new GameCharacter();
        public CharacterBuilder Named(string name)

```

```

    {
        _character.Name = name;
        return this;
    }
    public CharacterBuilder WithHealth(int health)
    {
        _character.Health = health;
        return this;
    }
    public CharacterBuilder Wielding(string weapon)
    {
        _character.Weapon = weapon;
        return this;
    }
    public CharacterBuilder WithSkill(string skill)
    {
        _character.Skills.Add(skill);
        return this;
    }
    public GameCharacter Build()
    {
        return _character;
    }
}
}

```

Usage:

Now, let's fluently create a game character with the CharacterBuilder:

```

using System;
namespace FluentInterfaceDesignPattern
{
    //Testing the Fluent Interface Design Pattern
    public class Program
    {
        public static void Main()
        {
            var hero = new CharacterBuilder()
                .Named("Aragorn")
                .WithHealth(100)
                .Wielding("Sword")
                .WithSkill("Leadership")
                .WithSkill("Swordsmanship")
                .Build();
            Console.WriteLine($"Name: {hero.Name}");
            Console.WriteLine($"Weapon: {hero.Weapon}");
            Console.WriteLine($"Skills: {string.Join(", ", hero.Skills)}");
            // Outputs:
            // Name: Aragorn
            // Weapon: Sword
            // Skills: Leadership, Swordsmanship
            Console.ReadKey();
        }
    }
}

```

```
}  
}
```

In this example:

- The CharacterBuilder provides a fluent interface to define a game character's properties.
- Methods such as Named, WithHealth, and Wielding allow for setting various character attributes, and they each return the builder instance for continued chaining.
- The Build method finalizes the character creation process and returns the constructed GameCharacter object.

This approach is highly readable and provides a structured way to create complex objects, like a game character, step by step. The Fluent Interface Design Pattern ensures the creation process remains intuitive and organized. When you run the above code, you will get the following output.

```
Name: Aragorn  
Weapon: Sword  
Skills: Leadership, Swordsmanship
```

Real-Time Example of Fluent Interface Design Pattern in C#: PizzaBuilder

A real-time example of the Fluent Interface Design Pattern can be found in libraries and frameworks designed to build or configure complex objects, conduct queries, or facilitate multi-step operations. One well-known example in C# is the StringBuilder class, but for this illustration, I'll present an example with a PizzaBuilder for a hypothetical pizza ordering application.

Let's say we're building an app to customize and order pizzas. With a Fluent Interface, you can design a PizzaBuilder that allows consumers to specify toppings, size, crust type, etc. Let us see how we can implement the above example using the Fluent Interface Design Pattern in C#:

```
using System.Collections.Generic;  
using System;  
namespace FluentInterfaceDesignPattern  
{  
    public class Pizza  
    {  
        public string Size { get; set; }  
        public string Crust { get; set; }  
        public List<string> Toppings { get; private set; } = new  
List<string>();  
        public override string ToString()  
        {  
            return $"{Size} inch {Crust} crust pizza with {string.Join(",  
", Toppings)}";  
        }  
    }  
    //Wrapper Class  
    public class PizzaBuilder  
    {  
        private Pizza _pizza = new Pizza();  
        public PizzaBuilder WithSize(string size)  
        {  
            _pizza.Size = size;  
            return this;  
        }  
    }  
}
```

```

    public PizzaBuilder WithCrust(string crust)
    {
        _pizza.Crust = crust;
        return this;
    }
    public PizzaBuilder AddTopping(string topping)
    {
        _pizza.Toppings.Add(topping);
        return this;
    }
    public Pizza Build()
    {
        return _pizza;
    }
}
// Testing the Fluent Interface Design Pattern
public class Program
{
    public static void Main()
    {
        PizzaBuilder builder = new PizzaBuilder();
        Pizza myPizza = builder.WithSize("12")
                                .WithCrust("thin")
                                .AddTopping("Mushrooms")
                                .AddTopping("Pepperoni")
                                .Build();

        Console.WriteLine(myPizza.ToString());
        Console.ReadKey();
    }
}

```

This design allows users to create a pizza by chaining methods together clearly and descriptively. The PizzaBuilder allows you to fluently design a pizza and produce a Pizza object using the Build() method.

This example makes complex object creation (like creating a custom pizza) more readable and intuitive. If you extend this example, you could add more customization options, like sauces, special instructions, etc., while maintaining a fluent and clear interface for the end developer. When you run the above code, you will get the following output.

12 inch thin crust pizza with Mushrooms, Pepperoni

Real-Time Example of Fluent Interface Design Pattern in C#: Entity Framework

Another prevalent and real-world example of the Fluent Interface Design Pattern in C# is the Entity Framework (EF), an Object-Relational Mapper (ORM) for .NET. One of its features, LINQ to Entities, allows you to write SQL-like queries directly in C# fluently. Here's a brief example demonstrating the fluent interface pattern with Entity Framework:

Creating the Model

```

using System;
namespace FluentInterfaceDesignPattern
{

```

```

public class Student
{
    public int StudentId { get; set; }
    public string Name { get; set; }
    public DateTime DateOfBirth { get; set; }
}

```

Using Entity Framework Fluent API to Configure Model Properties:

In Entity Framework, Fluent API provides a more advanced way to configure model classes when compared to Data Annotations. This API provides more functionality and is typically used in the `OnModelCreating` method of the context.

```

using System.Data.Entity;
namespace FluentInterfaceDesignPattern
{
    public class StudentDbContext : DbContext
    {
        protected override void OnModelCreating(DbModelBuilder
modelBuilder)
        {
            modelBuilder.Entity<Student>()
                .Property(s => s.Name)
                .IsRequired()
                .HasMaxLength(100);
        }
        public DbSet<Student> Students { get; set; }
    }
}

```

Performing a query using LINQ to Entities:

Suppose you want to retrieve all students aged 20 or above:

```

using System;
using System.Linq;
namespace FluentInterfaceDesignPattern
{
    // Testing the Fluent Interface Design Pattern
    public class Program
    {
        public static void Main()
        {
            using (var context = new StudentDbContext())
            {
                var students = context.Students
                    .Where(s => DateTime.Now.Year -
s.DateOfBirth.Year >= 20)
                    .OrderBy(s => s.Name)
                    .ToList();

                foreach (var student in students)
                {
                    Console.WriteLine(student.Name);
                }
            }
        }
    }
}

```



```

    }
    }
    Console.ReadKey();
}
}
}

```

In the example above:

- Where filters out students based on the condition provided.
- OrderBy sorts the result based on the Name property.
- Where and OrderBy methods return objects that allow for further chaining, making it a fluent interface.

Note that Entity Framework's use of Fluent Interface extends beyond just LINQ to Entities. The Fluent API, for instance, lets developers configure their models fluently without using attributes, as demonstrated in the OnModelCreating method.

Real-Time Example of Fluent Interface Design Pattern in C#: Fluent Email Builder

Let's delve into another context, focusing on configuring and sending emails, a frequent operation in many applications. Let us see how we can implement the above example using the Fluent Interface Design Pattern in C#:

Core Classes:

```

using System.Collections.Generic;
namespace FluentInterfaceDesignPattern
{
    public class Email
    {
        public string From { get; set; }
        public List<string> To { get; private set; } = new List<string>();
        public string Subject { get; set; }
        public string Body { get; set; }
        // ... Additional fields like CC, BCC, Attachments, etc.
    }
}

```

Fluent Email Builder:

```

using System;
namespace FluentInterfaceDesignPattern
{
    public class EmailBuilder
    {
        private readonly Email _email = new Email();
        public EmailBuilder From(string sender)
        {
            _email.From = sender;
            return this;
        }
        public EmailBuilder To(string recipient)
        {
            _email.To.Add(recipient);
            return this;
        }
    }
}

```

```

    }
    public EmailBuilder WithSubject(string subject)
    {
        _email.Subject = subject;
        return this;
    }
    public EmailBuilder WithBody(string body)
    {
        _email.Body = body;
        return this;
    }
    public Email Send() // This would be integrated with an actual
sending mechanism in a real-world scenario.
    {
        // Logic to send the email...
        Console.WriteLine($"Sent email to {string.Join(", ",
_email.To)} with subject '{_email.Subject}'.");
        return _email;
    }
}
}

```

Usage:

Now, let's fluently create and send an email using the EmailBuilder:

```

using System;
namespace FluentInterfaceDesignPattern
{
    //Testing the Fluent Interface Design Pattern
    public class Program
    {
        public static void Main()
        {
            var email = new EmailBuilder()
                .From("admin@dotnettutorials.net")
                .To("user1@dotnettutorials.net")
                .To("user2@dotnettutorials.net")
                .WithSubject("Welcome to Fluent Builder!")
                .WithBody("This is an example of the Fluent Interface Design
Pattern in C#.")
                .Send();
            Console.ReadKey();
        }
    }
}

```

In this example:

- The EmailBuilder offers a fluent interface to create an email.
- Methods like From, To, WithSubject, and WithBody configure various aspects of the email and return the builder instance for continuous chaining.
- The Send method finalizes the email creation and simulates sending the email in this mock example.

This approach ensures a clear, intuitive flow when setting up the email. The Fluent Interface Design Pattern simplifies the construction of complex objects, offering improved readability and a structured build process. When you run the above code, you will get the following output.

Real-Time Applications of the Fluent Interface Design Pattern:

The Fluent Interface Design Pattern primarily focuses on improving code readability and usability. By enabling method chaining to create a continuous flow of calls, it provides a clear and expressive way to perform various operations. Let's discuss a few real-time applications where the Fluent Interface shines.

ORM (Object-Relational Mapping):

Entity Framework (a popular ORM in .NET) uses a fluent API for configuring entity relationships and properties. This makes it easier to map complex entity relationships without resorting to attributes decorating the classes. For example:

```
modelBuilder.Entity<Student>()  
    .HasOne(s => s.Address)  
    .WithOne(ad => ad.Student)  
    .HasForeignKey<Address>(ad => ad.AddressId);
```

Builders for UI Components:

Some UI toolkits provide fluent interfaces to build complex UI components. Consider a hypothetical charting library:

```
chartBuilder.AddTitle("Sales Data")  
    .WithLegend(onTop: true)  
    .AddSeries(data)  
    .Render();
```

Configuration Systems:

Fluent interfaces can be used for configuring applications, especially when there's a need for setting up multiple configurations. This can be seen in middleware configurations in ASP.NET Core:

```
app.UseRouting()  
    .UseAuthentication()  
    .UseAuthorization()  
    .UseEndpoints(endpoints => { /* ... */ });
```

Test Frameworks:

FluentAssertions, a set of .NET extension methods, allows you to more naturally specify the expected outcome of a TDD or BDD-style test. Such as:

```
someString.Should().StartWith("Fluent").And.EndWith("Assertions");
```

Web API Clients:

Web API client libraries can benefit from fluent interfaces for building and executing requests. Think of a REST client where you can:

```
apiClient.Request("/endpoint")  
    .WithHeader("Authorization", "Bearer token")  
    .Post(data)
```

```
.ExpectStatus(HttpStatusCode.Created);
```

Query Builders:

Libraries like LINQ in C# use a form of the fluent interface to construct complex queries. This allows for a very expressive way to write queries in code.

```
var result = data.Where(x => x.Age > 20)
                  .OrderBy(x => x.Name)
                  .Select(x => x.Address);
```

StringBuilder in .NET:

The StringBuilder class allows for creating and manipulating string sequences more memory-efficiently than regular strings. Its methods return the StringBuilder instance, facilitating fluent usage.

```
var message = new StringBuilder()
               .Append("Hello, ")
               .Append("world!")
               .ToString();
```

LINQ in .NET:

LINQ's methods can be combined to filter, project, and transform collections.

```
var adultsNames = people.Where(p => p.Age >= 18)
                          .OrderBy(p => p.Name)
                          .Select(p => p.Name)
                          .ToList();
```

ASP.NET Core Middleware Configuration:

ASP.NET Core uses a Fluent Interface to configure middleware components in the application's pipeline.

```
app.UseRouting()
   .UseAuthentication()
   .UseAuthorization()
   .UseEndpoints(endpoints => {
       endpoints.MapControllers();
   });
```

Automapper:

Automapper is a popular object-object mapper in .NET. Its configuration can be achieved in a fluent manner.

```
var config = new MapperConfiguration(cfg => {
    cfg.CreateMap<Source, Destination>()
        .ForMember(dest => dest.SomeProperty, opt => opt.MapFrom(src =>
src.OtherProperty));
});
```

Fluent Validation:

This is a .NET library for fluently building strongly-typed validation rules.

This **is** a .NET library **for** building strongly-typed validation rules in a fluent manner.

```
public class CustomerValidator : AbstractValidator<Customer> {  
    public CustomerValidator() {  
        RuleFor(customer => customer.Name).NotEmpty().WithMessage("Name is  
required.");  
        RuleFor(customer => customer.Age).InclusiveBetween(18, 60);  
    }  
}
```

In each of these real-time applications, the Fluent Interface Design Pattern improves the readability and expressiveness of the code, guiding the developer through a set of operations logically and clearly.

In the next article, I will discuss the [Prototype Design Pattern in C#](#) with Examples. Here, in this article, I try to explain **Real-Time Examples of Fluent Interface Design Patterns in C#**. I hope you enjoy this Fluent Interface Design Pattern Real-Time Examples using C# article.